

Filter by protocol:

tcp
udp
http
dns
icmp

Filter by IP:

ip.src == 192.168.1.10
ip.dst == 192.168.1.20
ip.addr == 192.168.1.10 # source OR destination

Filter by port:

tcp.port == 80
tcp.srcport == 443
tcp.dstport == 22

Filter by MAC:

eth.addr == 94:44:52:ea:c6:d5

Combination filters:

ip.addr == 192.168.2.5 && tcp.port == 443
tcp && !http # TCP packets যেগুলো HTTP না

Apply a capture filter to capture only HTTP traffic

1. Wireshark open করো
2. Capture → Options (বা Ctrl+K চা পো)
3. "using this filter" box এ লিখো: tcp port 80
4. Interface select করো (তোমার active network)
5. Start button এ click করো
6. Browser এ কোনো HTTP site visit করো (http://example.com)
7. Stop capture
8. দেখবে শুধু port 80 এর packets আছে

Capture only DNS traffic

Capture filter:

udp port 53

বা:

port 53

Steps:

1. Wireshark → Capture Options
2. Filter: port 53
3. Start capture
4. Terminal এ: nslookup google.com
5. Stop capture
6. শুধু DNS packets দেখবে

Capture packets from a specific MAC address

Capture filter:

ether host 00:04:13:00:09:a3

বা নিজের MAC দিয়ে:

ether host aa:bb:cc:dd:ee:ff

Steps:

1. তোমার MAC address জানতে:
 - Windows: ipconfig /all
 - Linux: ifconfig
2. Capture filter এ: ether host YOUR_MAC
3. Start capture

Capture only TCP traffic, no UDP

Capture filter:

tcp

Steps:

1. Capture Options → Filter: tcp
2. Start capture
3. Browse websites (TCP traffic generate হবে)
4. ping করো (ICMP - capture হবে না)
5. nslookup করো (UDP DNS - capture হবে না)
6. Stop করে দেখো শুধু TCP packets আছে

Capture filter syntax

By Protocol:

tcp	# TCP packets only
udp	# UDP packets only
icmp	# ICMP packets only
arp	# ARP packets only
ip	# IPv4 packets only

By Host (IP):

host 192.168.1.10	# This IP (source or destination)
src host 192.168.1.10	# Source IP
dst host 192.168.1.10	# Destination IP

By Port:

port 80	# Port 80 (source or destination)
src port 80	# Source port
dst port 80	# Destination port
tcp port 80	# TCP port 80 only
udp port 53	# UDP port 53 only

By MAC:

ether host aa:bb:cc:dd:ee:ff	# Specific MAC
------------------------------	----------------

```
ether src aa:bb:cc:dd:ee:ff      # Source MAC
ether dst aa:bb:cc:dd:ee:ff      # Destination MAC
```

Logical Operators:

```
and          # Both conditions true
or           # Either condition true
not          # Negate condition
()           # Grouping
```

#Capture all HTTP traffic :

```
tcp port 80
or
port 80 (for both tcp and udp)
```

#Capture all HTTPS traffic :

```
tcp port 443
```

#DNS queries

```
udp port 53
udp dst port 53 (going to DNS server)
udp src port 53 (coming from DNS server)
```

#Exclude ssh

```
not tcp port 22
```

Capture unicast traffic only (no broadcast/multicast)
not broadcast and not multicast

#Capture web traffic (HTTP/HTTPS) to server 192.168.1.100
host 192.168.1.100 and (tcp port 80 or tcp port 443)

Capture traffic FROM subnet 172.16.0.0/24 TO server 10.0.0.100
src net 172.16.0.0/24 and dst host 10.0.0.100

TCP SYN Packets (Port Scan Detection)

Question: Capture TCP SYN packets (connection attempts)

```
tcp[13] & 2 != 0 and tcp[13] & 16 == 0
```

or

```
tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0
```

Capture email traffic (SMTP, POP3, IMAP)

```
tcp port 25 or tcp port 110 or tcp port 143
```

Port 25 = SMTP (sending)

Port 110 = POP3 (receiving)

Port 143 = IMAP (accessing)

Configure ring buffer to capture in multiple files

Steps:

1. Capture → Options
2. Output tab এ যাও
3. ☒ "Capture to a permanent file" enable করো
4. File browse করে location select করো
5. ☒ "Create a new file automatically" enable করো
6. Set করো:
 - "after 1000 kilobytes" (1MB)
 - "Use a ring buffer with 5 files"
7. Start capture

Result:

- 5টা file create হবে (file1.pcap to file5.pcap)
- Each file maximum 1MB
- 5th file full হলে 1st file overwrite হবে

Save captured packets automatically with timestamp

Steps:

1. Capture → Options → Output tab
2. ☒ "Capture to a permanent file"
3. File browse করে folder select করো
4. Filename লিখো: capture_%Y%m%d_%H%M%S
5. ☒ "Create a new file automatically"
6. Select: "after 60 seconds"
7. Start capture

Result:

Files create হবে timestamp সহ:

- capture_20260201_143052.pcap
- capture_20260201_143152.pcap
- capture_20260201_143252.pcap

এতে automatic timestamp হয়, কোন file কখন নেওয়া বুঝা যায়।

Display Filters Example :

1. Protocol Filters:

শুধু TCP packets:

tcp

শুধু UDP packets:

udp

শুধু HTTP packets:

http

শুধু DNS packets:
dns

ICMP packets:
icmp

ARP packets:
arp

TLS/SSL packets:
tls

2.IP address filters

Specific source IP:
ip.src == 192.168.1.10

Specific destination IP:
ip.dst == 192.168.1.20

Source OR destination (যেকোনো একটা):
ip.addr == 192.168.1.10

দুইটা specific IP এর মধ্যে:
ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20

Subnet filter:
ip.addr == 192.168.1.0/24

NOT this IP:
ip.addr != 192.168.1.10

3. Port filters

TCP port 80:
tcp.port == 80

Source port:
tcp.srcport == 443

Destination port:
tcp.dstport == 22

HTTP or HTTPS:
tcp.port == 80 || tcp.port == 443

Port range:
tcp.port >= 1000 && tcp.port <= 2000

UDP DNS:
udp.port == 53

4.TCP flag filters

SYN packets only:

```
tcp.flags.syn == 1
```

SYN but not ACK (connection initiation):

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

SYN-ACK packets:

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```

ACK packets:

```
tcp.flags.ack == 1
```

FIN packets (connection termination):

```
tcp.flags.fin == 1
```

RST packets (connection reset):

```
tcp.flags.reset == 1
```

PSH flag:

```
tcp.flags.push == 1
```

5.HTTP filters :

HTTP GET requests:

```
http.request.method == "GET"
```

HTTP POST requests:

```
http.request.method == "POST"
```

HTTP response code 200:

```
http.response.code == 200
```

HTTP errors (4xx, 5xx):

```
http.response.code >= 400
```

Specific website:

```
http.host == "www.google.com"
```

URI contains login:

```
http.request.uri contains "login"
```

HTTP traffic with specific User-Agent:

```
http.user_agent contains "Chrome"
```

6.DNS filters

All DNS queries:(not dns response)

```
dns.flags.response == 0
```

```
# All DNS responses:  
dns.flags.response == 1
```

```
# DNS query for specific domain:  
dns.qry.name == "google.com" or dns.qry.name contains "google.com"
```

```
# DNS queries containing "facebook":  
dns.qry.name contains "facebook"
```

```
# DNS A records:  
dns.qry.type == 1
```

```
# DNS response with IP:  
dns.a == 142.250.183.206
```

7.Packet Size Filters

```
# Packets larger than 1000 bytes:  
frame.len > 1000
```

```
# Packets smaller than 100 bytes:  
frame.len < 100
```

```
# Packets exactly 66 bytes:  
frame.len == 66
```

```
# Large packets (Jumbo frames):  
frame.len > 1500
```

```
#Show HTTP or HTTPS traffic
```

```
tcp.port==80 or tcp.port==443  
or  
tcp.port in {80 443}
```

8.Time Based Filters :

```
# Packets after specific time:  
frame.time >= "2026-02-01 14:30:00"
```

```
# Packets in time range:  
frame.time >= "2026-02-01 14:00:00" && frame.time <= "2026-02-01 15:00:00"
```

```
# First 100 packets:  
frame.number <= 100
```

9.Complex Combined Filters :

```
# HTTP traffic from specific IP:  
ip.src == 192.168.1.10 && http
```

TCP traffic excluding port 22 (SSH):

tcp && tcp.port != 22

All traffic except ARP and DNS:

!(arp || dns)

SYN packets to port 80:

tcp.flags.syn == 1 && tcp.dstport == 80

Large HTTP responses:

http.response && frame.len > 1000

Failed HTTP requests:

http.response.code >= 400 && http.response.code < 600

Show packets from network 172.16.0.0/1

ip.src==172.16.0.0/16

Find packets containing "password" in payload

frame contains "password" or http contains "password"

Show packets larger than 1000 bytes

Display Filter:

frame.len > 1000

Steps:

1. Capture complete করো
2. Filter: frame.len > 1000
3. শুধু বড় packets দেখাবে

Additional:

frame.len < 100 # Small packets

frame.len == 66 # Exact size

frame.len >= 1500 # Large packets

Show HTTP GET requests only

Display Filter:

http.request.method == "GET"

Steps:

1. Web browsing করার সময় capture করো
2. Filter apply করো
3. শুধু GET requests দেখবে

Related filters:

```
http.request.method == "POST"    # POST requests  
http.request.method == "PUT"     # PUT requests  
http.response.code == 200        # Successful responses
```

Use saved filters from the display filters window

Steps:

1. Analyze → Display Filters (বা Ctrl+Shift+D)
2. Saved filters এর list দেখবে
3. "HTTP" filter select করো
4. Double-click বা OK চা পো
5. Automatically "http" filter apply হবে

Custom Filter Save করা:

1. Display Filters window খোলো
2. "+" button click করো
3. Filter Name: "My Custom Filter"
4. Filter String: tcp.port == 8080
5. OK → Save হয়ে যাবে

Demonstrate the use of XOR operator

XOR (Exclusive OR) - শুধু একটা true হতে হবে, দুইটা না

Filter:

tcp xor udp

Meaning:

- Either TCP packets OR UDP packets
- But NOT both at the same time

Example scenario:

ip.addr == 192.168.1.10 xor ip.addr == 192.168.1.20

This will show:

- ✓ Packets from/to 192.168.1.10 only
- ✓ Packets from/to 192.168.1.20 only
- ✗ Packets between 192.168.1.10 and 192.168.1.20

Rarely used কিন্তু জানতে হবে!

Display Filters :

Protocol Filters:

tcp
udp
http
dns
icmp
arp
tls

```

# IP Filters:
ip.src == 192.168.1.10
ip.dst == 192.168.1.20
ip.addr == 192.168.1.10

# Port Filters:
tcp.port == 80
tcp.srcport == 443
tcp.dstport == 22

# TCP Flags:
tcp.flags.syn == 1      # SYN
tcp.flags == 18         # SYN-ACK
tcp.flags == 2          # SYN
tcp.flags == 16         # ACK
tcp.flags.fin == 1      # FIN
tcp.flags.reset == 1    # RST
tcp.flags.push == 1     # PSH

# Logical Combinations:
tcp && tcp.port == 80
ip.addr == 192.168.1.10 || ip.addr == 192.168.1.20
!arp && !dns
(tcp.port == 80) || (tcp.port == 443)

# Comparison:
frame.len > 1000
tcp.window_size < 1000
ip.ttl >= 64

# String Matching:
http.request.uri contains "login"
http.host == "www.google.com"
dns.qry.name contains "facebook"

```

Calculate TCP header size from a packet

Steps:

1. TCP packet select করো
2. Middle pane → Header Length field দেখো
3. Value সাধারণত 20 bytes (no options)
4. Options থাকলে বেশি হতে পারে

Example 1 - No options:

Header Length: 20 bytes (5 words × 4)
 $[5 \times 4 = 20]$

Example 2 - With options:

Header Length: 32 bytes (8 words × 4)
 Options: 12 bytes (32 - 20)

[MSS, Window Scale, Timestamps etc.]

Calculation:

Header Length field = number of 32-bit words

Actual bytes = Field value × 4

Minimum: 20 bytes

Maximum: 60 bytes

Identify retransmitted packets :

Retransmission কখন হয়:

- Packet loss
- ACK পাওয়া যায়নি
- Timeout হয়েছে

Wireshark Identification:

1. Wireshark automatically detects retransmissions
2. Info column এ লেখা থাকে: "[TCP Retransmission]"
3. Packet বাদামি color হয়

Filter:

tcp.analysis.retransmission

Example scenario:

Packet 100: Seq=1000, Len=500

(ACK পাওয়া গেল না)

Packet 150: [TCP Retransmission] Seq=1000, Len=500

(Same data আবার পাঠানো)

Causes:

- Network congestion
- Packet loss
- High latency

Filter to show only one TCP stream

Method 1 - Manual filter:

tcp.stream eq 5

Steps:

1. একটা TCP packet select করো
2. Middle pane → Expand "Transmission Control Protocol"
3. "Stream index" field দেখো (যেমন: 5)
4. Display filter: tcp.stream eq 5

5. Enter চা পো

Method 2 - Right-click:

1. **TCP packet এ right-click**
2. **"Conversation Filter" → TCP**
3. **Automatically filter apply হবে**

Result:

শুধু সেই connection এর packets দেখাবে,
SYN থেকে FIN পর্যন্ত সব।

Protocol Hierarchy:

Menu: Statistics → Protocol Hierarchy

TCP Stream:

Right-click → Follow → TCP Stream

Or: Analyze → Follow → TCP Stream

Common Filters:

tcp.stream eq 0 # First TCP connection
tcp.stream eq 1 # Second connection
!(tcp.stream eq 5) # Exclude stream 5
tcp.stream >= 10 # Streams 10 and above

Statistics Options:

- Protocol Hierarchy: Overall distribution
- Conversations: Pairwise communications
- Endpoints: Individual hosts
- I/O Graphs: Traffic over time

Frame time filter :

frame.time >= "2024-02-01 02:00:00" && frame.time <= "2024-02-01 03:00:00"

time = YYYY-MM-DD HH:MM:SS

TCP Flags (MUST KNOW):

Display Filter	Capture Filter	Hex
-----	-----	---
tcp.flags.syn==1	tcp[13] & 2 != 0	0x02

tcp.flags.ack==1	tcp[13] & 16 != 0	0x10
tcp.flags==0x02	tcp[13] == 2	SYN
tcp.flags==0x12	tcp[13] == 18	SYN-ACK
tcp.flags==0x10	tcp[13] == 16	ACK

ggh

Common Ports (Memorize!):

20/21 = FTP
22 = SSH
23 = Telnet
25 = SMTP
53 = DNS
80 = HTTP
110 = POP3
143 = IMAP
443 = HTTPS
3389 = RDP
8080 = HTTP Alternate